

05 动力学

作者 NILUSHANAN KULASINGHAM

阅读约 12 分钟 · 可交互

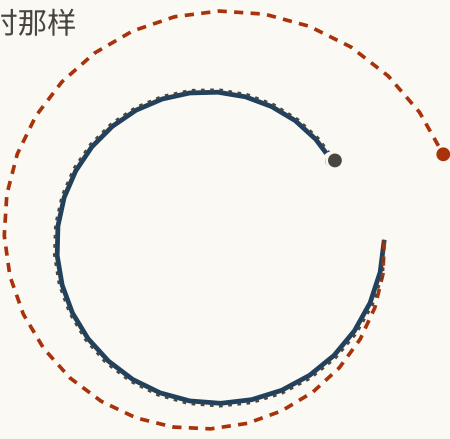
一个在训练时每一步都拿到真相的模型，一上线拿到的就是它自己上一次的答案。是什么把过去带着往前走，以及为什么那个头条准确率数字量的是另一份工作。

一个只往前预测一步的模型，很容易让人印象深刻。

把东西现在的位置给它，问它接下来会怎样，再把它的答案和实际发生的比一比。这么做一万次，读出平均误差。那个数字会很小，也会很诚实，而且它并不会告诉你你真正想知道的事。

因为你从来就不打算一次只用它一步。你打算问它一百步，而从第二步开始，它拿到的就不再是真相，而是它自己的答案。

实际发生的
每一步都被纠正，就像训练时那样
放它自己吃自己的答案



被纠正时，差了

0.016

自己跑时，差了

0.648

差了多少倍

39×

FIG. 5.1 同一个模型，同一个偏差，两种跑法。灰线在每一步都被纠正，就像训练时那样，而且它贴着真相贴得太紧，以至于基本消失在下面了。这正是结论。朱红色那条是同一个模型，被放开去吃自己的输出，也就是它将来真正会被使用的方式。

在每步 4% 误差、每步都被纠正的情况下，走三十步之后它仍然紧贴着真相。放开它自己跑，它最后差出去三十九倍。同一个模型。同一个错误。唯一变的，是每一步开始时交到它手里的东西。

那个错配

这件事有个名字，而这个名字对于一件这么简单的事来说，技术味重得没必要。训练的时候，序列模型通常在每一步都被展示真实的上一个值，因为记录里装的就是这个，而且这样训练更稳。这叫做**教师强制**。

麻烦在于，上线之后没有人会来给你做教师强制。真正使用的那一刻，模型拿到的是它自己上一次的答案，那个答案有点偏，而它从来没有被要求过从「有点偏」里恢复过来。

所以问题不是这个模型不擅长长时程。问题是它被训练在一个输入分布上，而你一开始往前推演，它就再也见不到那个分布了，训练循环里也没有任何东西注意到这一点。

再看一眼那张图，注意灰线真正在告诉你什么。它不是在衡量这个模型有多好。它是在衡量这个模型在一份它永远不会被要求做的工作上有多好。

每一步必须发生什么

把机器都剥掉，一个转移模型只做一件事：拿走你知道的，拿走你做了的，产出你接下来知道的。

别扭的是这三者里的第一个。你知道的不只是上一张画面。它是曾经发生过的一切相关的事，被折进一个小到能随身带的东西里。墙后面那个球还在动。你四个房间之前打开的那扇门还开着。

所以真正的问题是「过去是怎么被带着走的」，而这有两个答案。

带一份摘要。 保留一块固定大小的数字。每次有新东西来，就把它折进去，然后把旧的那块扔掉。不管序列变多长，每一步的工作量都不变。循环网络 (带环的网络：每一步的输出会被交回去，成为下一步输入的一部分) 做的就是这件事，最近重新流行起来的状态空间模型做的也是。

留住一个窗口。 把上下文装得下的步骤存起来；新的一步到来时，回头看它们，再决定什么相关。这就是注意力 (对每个新步骤，把当前可见的早先步骤按相关性打分，再主要从分高的那些里读)，也是 transformer 做的事。上下文是有限的。那次加权读取也会把可见的过去压成当前预测需要的东西。

	带一份摘要	留住上下文窗口
往前带的数字个数 下一步到来时，模型手里握着的东西。	256	32,768
多走一步的计算量 把一个新观测折进去所需的乘加次数。	65,536	32,768
它还看得到第一步吗？ 固定大小的摘要丢掉的东西，后面补不回来。	只有摘要留下了才行	还在上下文里时，可以看到
两列用的状态都是 256 个数字宽，所以唯一在变的只有长度。		

FIG. 5.2 两列用的状态宽度相同，所以唯一在变的只有序列有多长。两列都不是赢家。第二行展示了交叉点：对短序列来说，「全都看一遍」更便宜；对长序列来说则更贵。

底下的取舍不只是速度。一份固定摘要必须在每一步决定什么值得留下，而且当时还不知道以后会用到什么。注意力把这次决定推迟到读取时再做，代价是保留一组有限的早先状态。

这和摄像机与决定之间那道收窄口是同一种张力，只是高了一层。总得有东西被扔掉，而扔它的那个东西并不知道未来。

折中的办法

一个有用的技巧，值得知道，因为它出现在那些真正能用的系统里。

往前带两样东西，而不是一样。一部分每次都用同样的方式算出来，装的是从上一个状态可靠地推出来的东西；另一部分是采样出来的，装的是仍然可能走向任一边的东西。球的运动属于前者。门会不会开属于后者。

只留确定的那部分，模型就没法表示真正的不确定性，于是它会自信地编出一个未来。只留随机的那部分，它就很难把东西记很久，因为每一步都在注入噪声。PlaNet 主张两个都带，理由是：单独留下任何一个，失败的方向都不一样。

跟它共处

没有人消除掉这个错配。这个领域手里有的，是四种「不被它毁掉」的办法。

让它在自己的输出上训练。如果问题在于模型从来没练过从自己的错误里恢复，那就让它在训练时犯一些。有一部分时间喂它自己的预测，并且随着它变好把这个比例往上调。这件事通常叫 *计划采样*。

训练整段推演，而不是单步。别再给一步的准确率打分了，改成拿一整段想象出来的轨迹去和一整段真实的比，好让被优化的东西就是你真正要用的那个。

别推太远。让推演保持很短，并且从真实状态出发。便宜、有效，代价是放弃了当初那份「可以无限」的吸引力。

再看一眼。预测几步，执行其中一步，重新取一次观测，然后从头再来。机器人学里大半都是这个。一次真实测量能纠正状态里可见的部分；隐藏状态的误差和模型本身的偏差不会被清零。

这些都不是修好了。每一种办法都承认，学出来的转移模型只在有限时程内值得信任。工程的一部分，就是把那个时程测出来。

恢复是另一项本领

两个模型可以在通常的单步测试上打平，上线后却走向相反的结果。如果它们只见过演示轨迹，测试从来没有问过稍微走偏以后能不能回来。在受扰动或自己生成的状态上训练，才问到了漏掉的那道题。

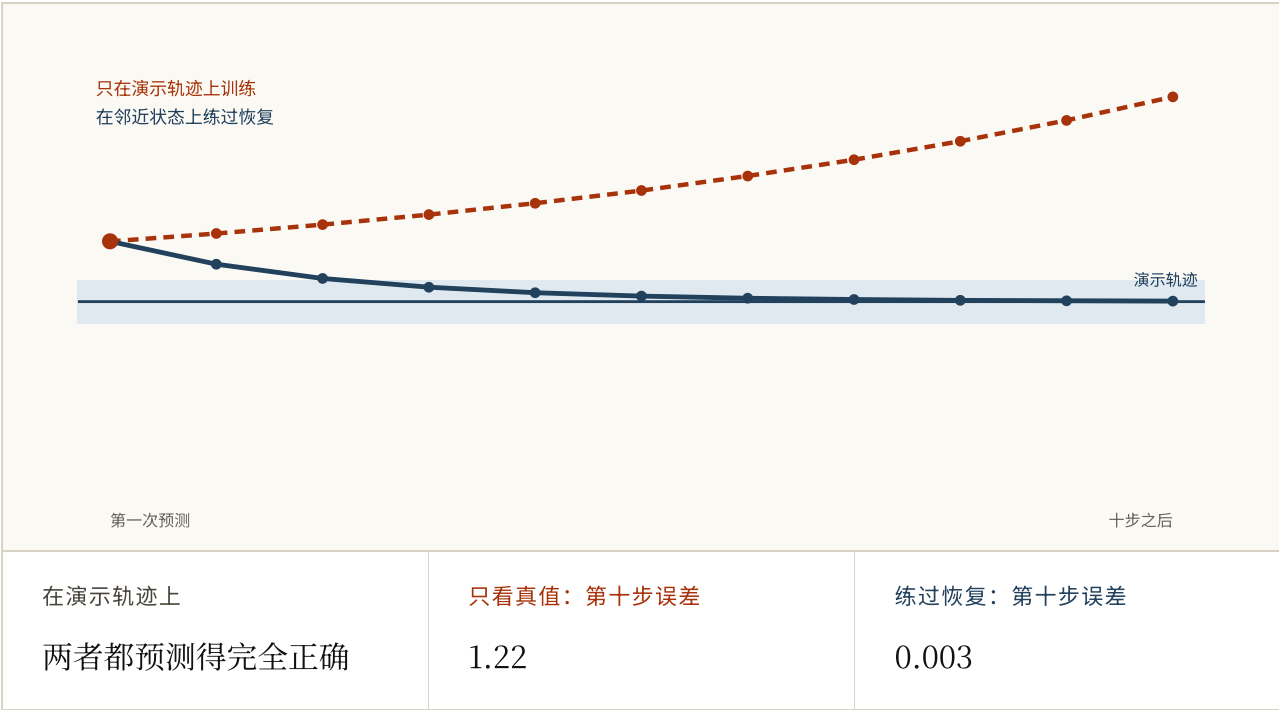


FIG. 5.3 两个玩具模型在演示轨迹上都完全正确。把起点移开：一个只学过线上的「下一步」，于是越走越远；另一个练过附近的状况，会弯回来。算术是示意，单步行为与自由运行时动力学之间的差别，正是 Professor Forcing 背后的实证问题。

计划采样会让学习者接触自己的状态，但它不保证结果。训练目标和上线时的分布仍在一起变化。Professor Forcing 从另一侧处理同一个缺口，让教师强制时的隐藏轨迹与自由运行时产生的轨迹相像。两种方法给出同一个结论：恢复能力必须在训练目标的某个位置出现。

试一试

1 一个转移模型报告说自己的平均单步误差非常低。为什么这个数字并不能告诉你你知道的事？

- a. 它多半量错了
- b. 你会一次用它很多步，而从第一步之后，喂给它的就是它自己的答案，不是真相
- c. 单步误差总是被低估
- d. 平均值掩盖了异常值

答案 b. 你会一次用它很多步，而从第一步之后，喂给它的就是它自己的答案，不是真相。这个测量是诚实的。它测的是一份这个模型永远不会被要求做的工作。

2 图里那条被纠正的线和那条自己跑的线，来自同一个模型、同一个每步偏差。为什么最后差这么远？

- a. 被纠正的那条用的是另一个模型
- b. 两次运行的随机噪声不同
- c. 自己跑的那条积累了更大的偏差
- d. 其中一条在每一步都被给了真实的上一个状态，所以它的错误从来没机会去喂任何东西

答案 d. 其中一条在每一步都被给了真实的上一个状态，所以它的错误从来没机会去喂任何东西。纠正会在每一步把输入重置回真相，于是误差没法复利。两条线之间，模型本身什么都没变。

3 什么是教师强制？

- a. 训练时每一步都把真实的上一个值展示给模型，因为记录里装的就是这个
- b. 强制模型使用固定学习率
- c. 只在最难的样本上训练
- d. 在更大的模型标注的数据上训练

答案 a. 训练时每一步都把真实的上一个值展示给模型，因为记录里装的就是这个。它让训练更稳，同时也意味着模型一次都没有被要求过从「有点偏」里恢复，而那恰恰是它之后唯一要做的事。

4 一段序列已经长过 transformer 的上下文窗口。哪句话准确？

- a. 序列变长只影响准确率，不影响计算量
- b. 注意力仍能逐字读取所有早先步骤
- c. 模型必须在窗口之外丢弃、压缩或检索；注意力只是推迟了这次决定
- d. 只有循环模型的记忆是有限的

答案 c. 模型必须在窗口之外丢弃、压缩或检索；注意力只是推迟了这次决定。注意力不必把过去挤进一个固定状态，但这只在有限上下文之内成立。为当前预测做的加权读取本身也在压缩可见步骤。

5 对于一个短序列来说，哪一种每步更便宜？

- a. 永远是注意力
- b. 注意力，直到序列长到越过交叉点为止
- c. 两者一样
- d. 永远是摘要

答案 b. 注意力，直到序列长到越过交叉点为止. 更新一份摘要的开销不随长度变化，所以它只有在序列够长时才占优。在交叉点以下，全都看一遍确实是更便宜的那个选项。

6 为什么要在状态里同时带一个确定的部分和一个随机的部分？

- a. 为了让模型可导
- b. 为了支持更大的批量
- c. 为了用更多参数
- d. 只有确定的部分表示不了真正的不确定性；只有随机的部分则很难记住东西，因为每一步都在注入噪声

答案 d. 只有确定的部分表示不了真正的不确定性；只有随机的部分则很难记住东西，因为每一步都在注入噪声. 单独留任何一个都会失败，而且失败的方向不同。球的运动属于前者；门会不会开属于后者。

7 两个模型在演示轨迹上的单步误差相同。受到一点扰动后，只有一个会回来。原来的测试漏掉了什么？

- a. 模型自己的误差或外界扰动产生的状态上的恢复行为
- b. 摄像机分辨率
- c. 转移是不是确定性的
- d. 模型的参数量

答案 a. 模型自己的误差或外界扰动产生的状态上的恢复行为. 只沿中心线做单步测试，永远不会访问上线后一次小错会产生状态。恢复是另一种行为，必须在那条线之外训练或检验。

8 计划采样、整段推演训练、短时程、重新规划，这四者的共同点是什么？

- a. 它们修好了这个错配
- b. 它们都让模型每一步更准
- c. 没有一个消除了这个错配；每一个都只是限制它能造成多大破坏
- d. 它们只适用于循环模型

答案 c. 没有一个消除了这个错配；每一个都只是限制它能造成多大破坏. 诚实的总结是：一个学出来的转移模型只在某个时程内值得信任，而工程的一部分就是知道那个时程有多长。

FIG. 5.4 八道关于这一步和那个错配的题。前三道会改变你读一篇论文头条数字的方式。

到这里为止，每一步都是用现实付的账。模型在录下来的经验上训练，在录下来的经验上检验，也被它纠正。

多谢阅读。第 6 章讲的是当你不再付账时会发生什么：模型好到一定程度，智能体就可以在它里面完成自己的学习，然后你会发现那个智能体在里面找到了什么。

参考资料

Professor Forcing: A New Algorithm for Training Recurrent Networks

LAMB ET AL., 2016 ↗

不直接安排模型吃多少自己的输出，而是让教师强制和自由运行时的隐藏轨迹彼此相像。恢复能力必须进入训练目标。

<https://arxiv.org/abs/1610.09038>

Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks

BENGIO ET AL., 2015 ↗

把这个错配点了名，并正面处理：训练时就让模型吃自己的预测，而且随着它变好逐步加量。

<https://arxiv.org/abs/1506.03099>

Sequence Level Training with Recurrent Neural Networks RANZATO ET AL., 2015

↗

给整段推演打分，而不是给单步打分，好让被优化的东西就是你真正要跑的那个东西。

<https://arxiv.org/abs/1511.06732>

Learning Latent Dynamics for Planning from Pixels (PlaNet)

HAFNER ET AL., 2018 ↗

从图像里学出随机潜在动力学，然后在决策时对它做搜索。图 2.4 的真实版本。

<https://arxiv.org/abs/1811.04551>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

在想象出来的潜在轨迹上训练 actor 和 critic，于是动手的那一刻不必再跑昂贵的搜索。

<https://arxiv.org/abs/1912.01603>

Long Short-Term Memory HOCHREITER & SCHMIDHUBER, 1997 ↗

那份固定摘要，被做成能比梯度愿意的更久地抓住一些东西。需付费。

<https://doi.org/10.1162/neco.1997.9.8.1735>

Attention Is All You Need VASWANI ET AL., 2017 ↗

另一个答案：别再做摘要了，把每一步都留着，代价是开销随长度增长。

<https://arxiv.org/abs/1706.03762>

Efficiently Modeling Long Sequences with Structured State Spaces (S4)

GU ET AL., 2021 ↗

「做摘要」这条路带着更好的机器回来了，也是状态空间模型重新进入讨论的原因。

<https://arxiv.org/abs/2111.00396>

Mamba: Linear-Time Sequence Modeling with Selective State Spaces

GU & DAO, 2023 ↗

一份会根据自己正在看的东西来决定留什么的摘要，而这正是固定版本做不到的那个让步。

<https://arxiv.org/abs/2312.00752>

FIG. 5.5 排序是给要在这上面接着做事的人用的，而不是为了历史完整性。